AI Usage Note

Phase 3 | Arian Vares

I uploaded project files as ZIPs so the AI could understand the work. The conversations shown here include explanations, checking the test scripts for possible errors, improving grammar and comment wording, and formatting the README files and failure-log PDF.

If needed, I can provide the original uploaded files for comparison with the GitHub versions but they are mostly the same AI only changed the styling the content is same .

CSCI3060U-DigitalGamesProject-mai...
File

i have uploaded all my files here use tham as sources first i want you to explain the difference between checking a program's terminal output, checking a file it creates, and checking its exit status use a small unrelated example, such as a calculator program explain how a program could produce the correct answer but still report an error do not write or modify my assignment scripts. Finish with three questions I should answer myself when reviewing my own test runner

When testing a program, there are three different things you can check: **terminal output, output files, and exit status**. Each one tells you something different about whether the program worked correctly.

1. Checking the terminal output

Terminal output is the text a program displays on the screen while running.

For example, imagine a calculator program that adds two numbers:

Plain text

```
Enter first number: 5
Enter second number: 3
Result: 8
```

A test runner can check whether the program displays `Result: 8`.

This confirms that the calculator printed the expected answer, but it doesn't necessarily mean the whole program worked correctly.

2. Checking a file the program creates

Some programs save results to files instead of only displaying them.

For example, the calculator might create a file called `result.txt` containing:

Plain text

```
8
```

A test runner can check whether the file exists and whether its contents match the expected result.

This is separate from checking terminal output. The calculator might display `8` correctly but accidentally save `0` in the file.

Ask ChatGPT Medium

So, **correct terminal output doesn't guarantee that the output file is correct**.

3. Checking the exit status

The exit status is a number the program returns to the operating system when it finishes.

Usually:

- **Exit status 0:** The program reports that it finished successfully.
- **Nonzero exit status:** The program reports an error or unsuccessful completion.

For example, the calculator could display:

```
Plain text
Result: 8
```

But still finish with:

```
Plain text
Exit status: 1
```

This means the calculator printed the correct result but reported that something went wrong.

How can a program produce the correct answer but still report an error?

Imagine the calculator successfully calculates `5 + 3 = 8` and displays the answer.

After printing the result, it tries to save the answer to `result.txt`. However, the program doesn't have permission to write to that file.

The terminal might display:

```
Plain text
Result: 8
Error: Could not save result.txt
```

The program then exits with status `1`.

The calculation was correct, but the entire operation wasn't successful because the file couldn't be saved.

A test that checks only `Result: 8` might miss this problem. Checking the output file and exit status would provide a more complete result.

This doesn't mean every test must check all three things. It depends on what the program is supposed to do

Ask ChatGPT Medium

How can a program produce the correct answer but still report an error?

Imagine the calculator successfully calculates `5 + 3 = 8` and displays the answer.

After printing the result, it tries to save the answer to `result.txt`. However, the program doesn't have permission to write to that file.

The terminal might display:

```
Plain text

Result: 8
Error: Could not save result.txt
```

The program then exits with status `1`.

The calculation was correct, but the entire operation wasn't successful because the file couldn't be saved.

A test that checks only `Result: 8` might miss this problem. Checking the output file and exit status would provide a more complete result.

This doesn't mean every test must check all three things. It depends on what the program is supposed to do.

Three questions to ask yourself when reviewing your test runner

1. **Terminal output:** Am I checking whether the program displays exactly what is expected, including error messages when necessary?
2. **Output files:** If the program creates a file, am I checking that the file exists and contains the correct information, rather than assuming the terminal output is enough?
3. **Exit status:** Does my test runner check whether the program finished with the expected exit status, even if its terminal output looks correct?

The main idea is that **producing the right answer and completing successfully are not always the same thing**. A reliable test runner checks the outputs and completion status that matter for each test.

help me check my understanding of command-line arguments in C++ use a fictional image conversion program rather than my project explain how the executable name contributes to argc, how filenames containing spaces are passed, and how command-line arguments differ form redirected standard input do not give me assignment code.

Take a fictional C++ program called `imgconv` that converts an image from one format to another.

1. The executable name and `arg`

In C++, command-line arguments are received through `main`, usually using `argc` and `argv`.

Ask ChatGPT Medium

1. The executable name and `argc`

In C++, command-line arguments are received through `main`, usually using `argc` and `argv`.

- `argc` tells us how many arguments were passed, including the program name.
- `argv` stores those arguments as an array of C-style strings.

For example:

```bash
./imgconv photo.png photo.jpg
```

The arguments would be:

Position	Value
`argv[0]`	`./imgconv`
`argv[1]`	`photo.png`
`argv[2]`	`photo.jpg`

So `argc == 3`, even though the user provided only two filenames.

The reason is that the executable name also counts as an argument.

The program can also receive numbers as arguments, but they initially arrive as strings. For example, `--quality 80` passes `"80"` as text, not as an integer.

One other detail is that `argv[argc]` is always a null pointer, marking the end of the argument array.

2. Filenames containing spaces

The shell normally treats spaces as separators between arguments.

Imagine the image is named `my holiday.png`.

If the user writes:

```bash
./imgconv my holiday.png out.jpg
```

The program receives:

Position	Value
`argv[0]`	`./imgconv`

Ask ChatGPT Medium

The program receives:

Position	Value
`argv[0]`	`./imgconv`
`argv[1]`	`my`
`argv[2]`	`holiday.png`
`argv[3]`	`out.jpg`

Here, `argc == 4`.

The filename was split into two arguments, which probably isn't what the user intended.

To keep a filename containing spaces together, the user can put it in quotation marks:

```bash
./imgconv "my holiday.png" out.jpg
```

Or escape the space:

```bash
./imgconv my\ holiday.png out.jpg
```

Both commands produce the same arguments:

- `argv[0]` = `./imgconv`
- `argv[1]` = `my holiday.png`
- `argv[2]` = `out.jpg`

Now `argc == 3`.

The shell processes the quotation marks or backslash before passing the arguments to the program. The program receives the filename without the quotation marks.

3. Command-line arguments vs. redirected standard input

These are two different ways of providing information to a program.

Command-line arguments are provided when the program starts. They are available through `argc` and `argv`.

Standard input is a stream of data that the program reads while running, usually through `std::cin` or `std::getline`.

For example:

Ask ChatGPT Medium

Standard input is a stream of data that the program reads while running, usually through `std::cin` or `std::getline`.

For example:

```bash
./imgconv photo.png photo.jpg < settings.txt
```

In this command:

- `photo.png` is a command-line argument.
- `photo.jpg` is another command-line argument.
- `< settings.txt` tells the shell to use the contents of `settings.txt` as standard input.

The important detail is that `settings.txt` **does not become an argument**.

The program still receives:

```plaintext
argc = 3

argv[0] = ./imgconv
argv[1] = photo.png
argv[2] = photo.jpg
```

Instead, the program can read the contents of `settings.txt` through standard input.

The same general rule applies to output redirection using `>` or `2>`: those redirections are handled by the shell and do not add arguments to `argv`.

Main difference

Command-line arguments	Redirected standard input
Passed when the program starts	Read while the program runs
Accessed through `argc` and `argv`	Accessed through input operations such as `std::cin`
Commonly used for filenames, options and short values	Commonly used for input data
Count toward `argc`	Do not count toward `argc`

The main idea is that command-line arguments tell the program what to work with, while standard input supplies data the program can read during execution.

Quiz

Ask ChatGPT Medium

The main idea is that command-line arguments tell the program what to work with, while standard input supplies data the program can read during execution.

Quiz

Question 1. Imagine someone runs this command:

```bash
./imgconv "summer trip 2026.png" thumb.jpg --quality 80 < batch.txt
```

What is the value of `argc`, and what string is stored in `argv[3]`?

Explain how you counted the arguments, including why `batch.txt` does or doesn't count.

open phase3-frontend-testing/scripts/run_all_tests.sh. review the section that builds summary.txt, especially how WHY, CODE, and WANTED determine PASS or FAIL could a test pass despite an unexpected exit status, missing comparison result, or helper-script failure? explain the control flow and any assumptions. do not modify the script.

A test cannot pass if its exit status differs from the expected exit status. However, **a test can incorrectly pass if its comparison result is missing or if a helper script fails without reporting the failure properly**.

I reviewed `run_all_tests.sh`, `run_tests.sh`, `compare_daily.sh`, and `compare_terminal.sh` from the uploaded project ZIP. No files were modified.

1. How the test runner works

The main script, `run_all_tests.sh`, follows five steps:

1. **Build:** Unless `FRONTEND` is already set, it runs `make` to build the program. If building fails, the script exits with status `2`.
2. **Run tests:** `run_tests.sh` runs the program for each test and saves its terminal output (`.out`), daily transaction file (`.atf`), standard error (`.err`), exit status (`.code`), and test group (`.group`).
3. **Compare results:** `compare_daily.sh` and `compare_terminal.sh` compare actual output against expected files. Their results are saved in `daily_check.txt` and `terminal_check.txt`.
4. **Create summary:** The script examines the saved exit codes and comparison results to decide whether each test passes or fails.
5. **Finish:** It generates a report and exits with status `1` if any counted test failed, or `0` otherwise.

2. How `WHY`, `CODE`, and `WANTED` determine PASS or FAIL

Ask ChatGPT Medium

test passes or fails.

5. **Finish:** It generates a report and exits with status `1` if any counted test failed, or `0` otherwise.

2. How `WHY`, `CODE`, and `WANTED` determine PASS or FAIL

The main part of the summary logic uses three variables.

`WHY` — reasons a test failed

At the beginning of each test's summary check:

```bash
WHY=""
```

The script adds a reason when it finds a failed comparison:

```bash
if grep -q "^FAIL $NAME\$" "$RUN/daily_check.txt"; then
    WHY="$WHY daily"
fi

if grep -q "^FAIL $NAME\$" "$RUN/terminal_check.txt"; then
    WHY="$WHY terminal"
fi
```

If the daily file comparison fails, it adds `daily`. If the terminal comparison fails, it adds `terminal`.

`CODE` — actual program exit status

```bash
CODE="$(cat "$FILE")"
```

This reads the exit status saved when the test program ran.

`WANTED` — expected program exit status

```bash
WANTED=0
```

By default, the script expects exit status `0`. If the test has an `expected_code.txt`, it uses that value instead.

Ask ChatGPT Medium

By default, the script expects exit status `0`. If the test has an `expected_code.txt`, it uses that value instead.

The comparison is:

```bash
if [ "$CODE" != "$WANTED" ]; then
    WHY="$WHY exit_$CODE(wanted_$WANTED)"
fi
```

If the actual and expected exit statuses are different, the script adds another failure reason.

Finally:

```bash
if [ -z "$WHY" ]; then
    echo "PASS $NAME $GROUP" >> "$SUMMARY"
else
    echo "FAIL $NAME $GROUP$WHY" >> "$SUMMARY"
fi
```

The important detail is that **a test passes when `WHY` is empty**.

3. Could a test pass with an unexpected exit status?

Under the summary logic, no.

For example, suppose a test produces the correct terminal output and daily file, but exits with status `1` when the expected status is `0`.

The script would record:

```plain text
FAIL example_test example exit_1(wanted_0)
```

This means the correct output alone is not enough to pass the test.

A nonzero exit status can still be correct if the test explicitly expects it through `expected_code.txt`.

One assumption is that tests without an `expected_code.txt` should exit with status `0`. This applies even to tests that intentionally supply invalid input.

The exit-status comparison uses strings rather than nu ical comparison, so differences in formatting can also cause a failure.

Ask ChatGPT Medium

cause a failure.

4. Could a test pass if a comparison result is missing?

Yes. This is an important weakness in the current logic.

The script checks whether the comparison files contain:

```
Plain text
FAIL test_name
```

But it does not verify that they contain:

```
Plain text
PASS test_name
```

For example, imagine a test has the correct exit status, but `compare_terminal.sh` unexpectedly fails to write either a PASS or FAIL result for that test.

The summary logic would not find a `FAIL` line. If the daily comparison also has no failure and the exit status matches, `WHY` would remain empty.

The test would therefore be marked as PASS, even though its terminal output was never confirmed as correct.

The individual comparison scripts normally detect missing output files and report failures. The problem appears when a comparison result itself is skipped or not produced.

5. Could a helper-script failure be missed?

Yes, depending on how the helper fails.

The main script checks helper exit statuses differently.

`run_tests.sh`

If it returns a nonzero status, `run_all_tests.sh` stops with exit status `2`.

`compare_daily.sh` **and** `compare_terminal.sh`

The main script stops only when either comparison helper returns a status greater than `1`.

This is because the comparison helpers normally use:

- `0` when comparisons pass.
- `1` when tests fail.
- `2` for certain script errors, such as invalid argume...

Ask ChatGPT Medium

- `2` for certain script errors, such as invalid arguments.

The issue is that a helper could fail unexpectedly but still exit with `0` or `1`.

For example, if a comparison script exits with `1` before writing its results, the main script treats that status as an ordinary test failure rather than a broken helper.

If no `FAIL` lines were written, the summary could incorrectly mark tests as passing.

The script assumes that exit status `1` from a comparison helper means its results were successfully produced and contain the relevant failed tests.

6. What if a test never runs?

The summary loop processes the `.code` files:

```bash
for FILE in "$OUT"/*.code; do
```

It does not directly iterate through the original test folders.

This means a test that never produces a `.code` file is not individually counted as a failed test.

There is also no explicit comparison between the number of test folders and the number of completed tests.

If no `.code` files exist at all, the unmatched wildcard can cause the loop to process the literal `*.code` path, producing an accidental failure. However, this is not an intentional check that ensures every expected test ran.

Another assumption is that helper commands such as `cat` and `grep` work correctly. The script does not use `set -e`, `set -u`, or `pipefail`, so errors from individual commands do not automatically stop the script.

7. Summary

Situation	Can it incorrectly pass?	Reason
Exit status differs from expected	No	Adds an `exit_...` failure reason
Missing terminal comparison result	Yes	No `FAIL` line is found
Missing daily comparison result	Yes	No `FAIL` line is found
Comparison helper exits with status greater than 1	No	Main script stops with status 2
Comparison helper fails but exits 0 or 1	Yes	Its failure may not be represented in the summary
A test never produces a `.code` file	Yes, it can go uncoun	Summary only processes saved `.code` files

Ask ChatGPT Medium

uncounted

Conclusion

The script correctly checks whether the program's actual exit status matches the expected status. It also records failures reported by the terminal and daily-file comparison scripts.

However, **the summary relies on finding explicit `FAIL` lines rather than requiring proof that every comparison passed**.

This creates a possible false PASS when a comparison result is missing or when a helper script stops without properly recording its results.

The main assumption is that every test produces its expected result files and that each comparison helper completes its work and reports every test correctly.

CSCI3060U-DigitalGamesProject-mai...
File

i have uploaded our work zip folder everything that we have done is there this is only for phase 3 folder first i want you to read all the readme.md files for phase 3 and the one in the root dont change any meaning or data if anythings look wrong just let me know but dont fix anything your self at any cost for the readme files take watever i wrote and make them look good the readme in the files i sent you is in pdf so create the tables and make them look good for git also if you see any grammer or spelling errors fix them dont need to tell me about them also its ok to make it little better but again dont change any meaning or anything you job is only to make it look good also there is a pdf in the file called phase3_failure_log i want you to first read the finall pdf for the phase 2 in the zip i sent then create phase3 pdf exactly in the same style and tables and format as phase2 pdf also again dont change anything (this is a hard rule) only thing you can fix and change is grammer and spelling errors
also both for pdf and reame files if there is a sentence that can be wrote better write the updated verision in front of it in bold red i will decide later to use it or not send all files back in a zip and dont explain

Phase3.zip ↗

Ask ChatGPT Medium

for the readme in the root add all who did what part in one table and for all phase 1,2,3 so its easier to read also also the.markdown for phase 2 looks wired make it like phase 1 and 3.
also in readme in phase 3 folder add this before decision part :

README.zip ↗

read eveyfile in phase 3 nad read the comments if there is anything wrong or wierd in anyway dont change or fix them just write4 your better version in red bold in front of it i will decide to use them or not if there isnt a need to change or make it more detailed just ignore and let me know if changed just give me the files back

Comments.zip ↗

for the pdf of phase 3 make it look a lot more than phase 2 like they are same file also put all commands in a markdown also add all the red bold notes i added to the pdf they are the updates dont change anything else only send me the file and dont explain

Phase3_Failure_Log.pdf ↗

Phase3_Failure_Log.pdf

Ask ChatGPT

Medium